



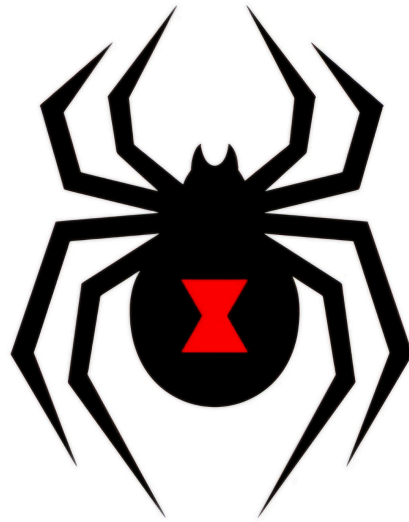
UNSA

Universidad Nacional de San Agustín

“AÑO DE LA ESPERANZA Y EL FORTALECIMIENTO DE LA DEMOCRACIA”

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



VIUDA NEGRA

Plan de Pruebas Unitarias

ASIGNATURA:

Pruebas de Software

DOCENTE:

Ing. Robert Edison Arisaca Mamani

INTEGRANTES:

Tejada Lazo Jordy Rolando (Test Lead)

Hurtado Bejarano Michael Steve (Test Analyst)

Jaita Chura Jose Manuel (Test Design)

Yare Chulunquia Kevin Pedro (Test Analyst)

Del Castillo Montoya Christopher Brad (Architect)

Índice

Índice	2
1. Introducción	4
1.1. Alcance	4
1.2. Referencias	4
1.3. Glosario	5
2. Contexto de las Pruebas	5
2.1. Descripción del Proyecto InvenTree	5
2.2. Descripción Funcional y Arquitectónica de Módulos del Sistema	6
2.3. Elementos de Prueba (Scope Técnico)	9
2.4. Alcance de la Prueba	9
2.5. Suposiciones y Restricciones	9
3. Comunicación de las Pruebas	10
3.1. Protocolo de Comunicación	10
3.2. Puntos de Sincronización Estructurados	10
4. Registro de Riesgos Técnicos	11
5. Estrategia de Prueba Unitaria	13
5.1. Subprocesos de prueba	13
5.1.1. Backend (Python/Django Core)	13
5.1.2. Frontend (Lógica Atómica)	13
5.1.3. Pruebas de Integración de API (Contexto de Suite)	14
5.2. Estructura Organizacional de la Suite	14
5.3. Catálogo de Comandos de Automatización	14
5.4. Variables de Entorno Obligatorias para Pruebas	16
5.5. Técnicas de Diseño Aplicadas en la Suite	16
5.6. Criterios de Finalización del Sprint	16
5.7. Métricas de Control de Calidad	17
6. Actividades y Estimados de Prueba	17
7. Requisitos y Herramientas del Entorno de Pruebas	18
7.1. Ambiente de Infraestructura Local	18
7.2. Frameworks y Herramientas Técnicas	18

INVENTREE Plan de Pruebas Unitarias	UNSA - EPIS Página 3
--	---------------------------------------

Control de versiones

Versión	Autor(es)	Descripción	Fecha
1.0	Hurtado Bejarano Michael Steve	Versión inicial del documento	05/06/26
2.0	Hurtado Bejarano Michael Steve	Complementar información de los respectivos módulos del programa así como su funcionamiento	20/06/2026

INVENTREE Plan de Pruebas Unitarias	UNSA - EPIS Página 4
--	---------------------------------------

1. Introducción

1.1. Alcance

El presente plan define el marco formal para planificar, gestionar y ejecutar las pruebas UNITARIAS del sistema InvenTree v1.3.0 (Open-Source under MIT License). El foco de este documento es estrictamente la capa de validación atómica y aislada del backend (Python/Django) y del frontend (React/TypeScript), cubriendo los módulos críticos del sistema de gestión de inventario.

InvenTree ya cuenta con una suite de pruebas robusta y preexistente en su repositorio oficial (github.com/inventree/InvenTree). El objetivo primordial de este plan es RE-EJECUTAR dichas pruebas, comprender a fondo su estructura lógica, verificar que la cobertura de código combinada se mantenga en los estándares del proyecto (~87%) y documentar el proceso dentro de las pautas académicas del curso.

1.2. Referencias

- Repositorio oficial del software: <https://github.com/inventree/InvenTree>
- Documentación oficial de ingeniería: <https://docs.inventree.org/en/latest/>
- Workflow central de CI/CD: [.github/workflows/qc_checks.yaml](https://github.com/inventree/InvenTree/blob/master/.github/workflows/qc_checks.yaml) (rama master)
- Estándar Internacional para Pruebas de Software: ISO/IEC/IEEE 29119
- Django Testing Framework Docs: <https://docs.djangoproject.com/en/stable/topics/testing/>
- Pytest-Django Framework Integration: <https://pytest-django.readthedocs.io/en/latest/>
- Tablero oficial de cobertura de código: <https://app.codecov.io/gh/inventree/InvenTree>
- Archivo de referencia: PLANTILLA - Plan_de_Pruebas.docx (Curso Pruebas de Software, UNSA-EPIS)

1.3. Glosario

- **PU (Prueba Unitaria):** Técnica de verificación que inspecciona unidades mínimas de código fuente (funciones, métodos o clases) de forma completamente aislada.
- **TDD (Test-Driven Development):** Metodología de desarrollo guiada por la escritura previa de pruebas.
- **CI/CD (Continuous Integration / Continuous Deployment):** Automatización del proceso de integración y despliegue de código.
- **REST API:** Interfaz de programación basada en el protocolo de transferencia de estado representacional (implementada vía Django REST Framework).
- **ORM (Object-Relational Mapping):** Capa de abstracción encargada de mapear los modelos orientados a objetos con las tablas relacionales de la base de datos.
- **Fixture:** Set de datos predefinidos o estados de base de datos estáticos utilizados para inicializar el contexto reproducible de un test.
- **Coverage:** Métrica porcentual que denota la cantidad de líneas de código ejecutadas durante la corrida de una suite de pruebas.
- **Invoke:** Herramienta de automatización de tareas en Python utilizada mediante un archivo de configuración tasks.py en InvenTree.
- **Mock / Stub:** Dobles de prueba u objetos simulados que reemplazan dependencias externas complejas (APIs, archivos, red) para aislar la unidad bajo prueba.
- **SQLite:** Motor de base de datos relacional ligero en memoria utilizado por defecto en el entorno de pruebas local.
- **QC (Quality Check):** Nombre técnico asignado al pipeline automatizado principal de control de calidad en GitHub Actions para InvenTree.

2. Contexto de las Pruebas

2.1. Descripción del Proyecto InvenTree

InvenTree es un sistema open-source maduro de gestión de inventarios enfocado en el control de stock de bajo nivel y la trazabilidad detallada de partes y componentes.

Atributo	Detalle técnico Basal
Versión bajo Análisis	v1.3.0 (Release: Abril 2026)
Licencia	MIT License

INVENTREE Plan de Pruebas Unitarias	UNSA - EPIS Página 6
--	---------------------------------------

Arquitectura Backend	Python 3.11 / 3.14 + Django 5.x + Django REST Framework (DRF)
Arquitectura Frontend	React + TypeScript + Mantine UI + TanStack Query
Motores de Persistencia	SQLite (entorno local/test), PostgreSQL 17 / MySQL 9 (entornos productivos)
Mapeo de Tareas Async	Django Q2 (Manejo en segundo plano de reportes y notificaciones automáticas)
Orquestador de CI/CD	GitHub Actions mediante el script unificado qc_checks.yaml (681 líneas)
Métricas de Cobertura	~87% combinado reportado oficialmente en el badge de Codecov
Métricas del Repositorio	+7.1k estrellas, 1.4k forks y 17,657 commits históricos

2.2. Descripción Funcional y Arquitectónica de Módulos del Sistema

Para establecer un contexto claro sobre los elementos que serán evaluados en esta suite de pruebas, la arquitectura backend de InvenTree (desarrollada bajo el framework Django) se divide en aplicaciones lógicas según su responsabilidad. A continuación se detallan los módulos, clasificados por su nivel de impacto en el sistema:

A. Módulos Centrales de Negocio (Core Apps)

- **Módulo part (Catálogo Maestro y Listas de Materiales):**
Gestiona la abstracción de los artículos. No controla existencias físicas, sino el "concepto" del producto (ej. especificaciones técnicas, variantes, parámetros). Es crítico porque administra las Listas de Materiales (BOM - Bill of Materials), definiendo las dependencias exactas y cantidades requeridas de componentes para fabricar subensamblajes o productos terminados.
- **Módulo stock (Control de Inventario y Trazabilidad Físico):**
Responsable de la materialización de las "Parts". Gestiona las existencias reales mediante el seguimiento de cantidades, ubicaciones espaciales jerárquicas (almacén > estante > caja), números de serie y lotes. Registra todo el historial transaccional de movimientos (entradas, salidas, transferencias y conteos cíclicos).

- **Módulo build (Órdenes de Producción y Ensamblaje):**
Administra el flujo de manufactura. Orquesta la interacción entre el módulo part y stock tomando una receta (BOM) para crear Órdenes de Trabajo. Su lógica controla la reserva de material (allocations), el consumo de stock de las ubicaciones y el ingreso automático del producto terminado al inventario.
- **Módulo order (Logística Transaccional):**
Gestiona el flujo comercial del inventario hacia el exterior. Controla las Órdenes de Compra (Purchase Orders) para el reabastecimiento de componentes y las Órdenes de Venta (Sales Orders) para el despacho de productos terminados. Incluye máquinas de estado complejas para rastrear envíos pendientes, recepciones parciales y devoluciones.
- **Módulo company (Directorio de Entidades Comerciales):**
Funciona como el CRM interno del sistema. Clasifica a las entidades externas en tres grandes grupos: Proveedores (Suppliers), Fabricantes (Manufacturers) y Clientes (Customers). Se relaciona directamente con el módulo order para definir los actores de cada transacción y con el módulo part para establecer números de parte de fabricantes externos.

B. Módulos de Soporte al Usuario (Support Apps)

- **Módulo users (Gestión de Identidad y Accesos):**
Implementa la seguridad del sistema mediante Control de Acceso Basado en Roles (RBAC). Gestiona la autenticación, grupos de usuarios, generación de tokens para la API REST y los permisos a nivel de objeto/modelo para restringir qué acciones puede realizar cada usuario.
- **Módulo report (Motor de Generación de Documentos):**
Encargado del renderizado dinámico de archivos PDF. Transforma los datos de la base de datos (facturas, albaranes, reportes de pruebas de calidad, órdenes de compra) en documentos listos para impresión mediante el uso de plantillas predefinidas basadas en HTML/CSS.
- **Módulo plugin (Arquitectura de Extensibilidad):**
Define la interfaz y el ciclo de vida para complementos de terceros. Permite que el sistema core interactúe de forma segura con aplicaciones externas (como sistemas ERP, pasarelas de pago o

escáneres de hardware) mediante "hooks", sin necesidad de alterar el código fuente original.

- Módulo machine (Integración M2M - Machine to Machine):
Provee los endpoints y el estado necesario para la comunicación directa con maquinaria automatizada en entornos industriales, permitiendo a máquinas de ensamblaje (ej. Pick-and-Place) leer el inventario o reportar consumos mediante API.

C. Módulos de Infraestructura y Utilidades (Technical/Utility Apps)

- Módulo importer / data_exporter (Procesos ETL):
Encapsulan la lógica de Extracción, Transformación y Carga (ETL) del sistema. Manejan el parseo masivo, validación de integridad y mapeo de columnas cuando los usuarios suben archivos (CSV/Excel) para poblar la base de datos o cuando requieren extraer volcados de información.
- Módulo common (Configuraciones y Tareas Asíncronas):
Centraliza las variables de entorno, configuraciones globales del sistema (moneda por defecto, métricas, unidades), gestión de webhooks para disparar eventos externos, notificaciones y la definición de tareas asíncronas en segundo plano (vía Django Q o Celery).
- Módulo generic (Plantillas de Abstracción):
No expone funcionalidades al usuario final. Aplica el principio DRY (Don't Repeat Yourself) alojando clases base abstractas (Abstract Models), mixins y serializadores genéricos que son heredados por el resto de módulos para estandarizar comportamientos (ej. auditoría de campos genéricos de creación/modificación).
- Módulo web (Enrutamiento e Interfaz):
Gestiona las vistas renderizadas en el servidor, la configuración de URLs principales de la aplicación web tradicional, el manejo de sesiones del navegador y los middlewares encargados de servir la interfaz gráfica inicial antes de delegar el control al framework frontend.

2.3. Elementos de Prueba (Scope Técnico)

- **Backend (Python/Django Core):** Validación de modelos ORM, métodos de cálculo internos, señales (signals) de Django, restricciones de persistencia, serializadores de Django REST Framework y ViewSets lógicos.
- **Base de Datos (Migraciones):** Pruebas de consistencia de esquemas estructurales al aplicar o revertir migraciones de datos.
- **Frontend (Lógica Pura):** Pruebas lógicas de funciones helper independientes, hooks personalizados y formateadores de datos mediante Vitest.
- **API REST:** Verificación técnica de los endpoints CRUD centrales mediante aserciones con APITestCase de DRF.

2.4. Alcance de la Prueba

- **Incluido en el alcance:**
 - Re-ejecución total y controlada de la suite de pruebas unitarias provista en InvenTree.
 - Control del umbral mínimo de cobertura de código para que se mantenga por encima o igual al 85% global (referencia base del proyecto: ~87%).
 - Validación de reglas de negocio en los modelos centrales (Part, StockItem, StockLocation, Order, Build).
 - Pruebas lógicas sobre el disparo correcto de señales de auditoría (ej. actualización automática de historial al modificar stock).
 - Análisis e interpretación del pipeline automatizado en GitHub Actions (qc_checks.yaml).
- **Excluido del alcance:**
 - Pruebas de integración complejas entre módulos (delegadas al Hito 3).
 - Benchmarks de rendimiento de hardware o tiempos de respuesta bajo estrés extremo.
 - Pruebas de interfaz gráfica de usuario de extremo a extremo (E2E) con Playwright o Selenium.
 - Auditorías de seguridad perimetral avanzado (Penetration Testing) o configuración de orquestadores de contenedores en producción (Kubernetes).

2.5. Suposiciones y Restricciones

- **Suposiciones:**
 - Cada miembro del equipo cuenta con entornos locales limpios con Python 3.11+ y Node.js 20+ instalados.

- Se trabajará de manera unificada clonando la rama estable master del repositorio oficial.
- La ejecución local se procesará con SQLite en memoria, emulando fielmente el comportamiento de aislamiento de los jobs rápidos del CI.
- El task runner pyinvoke está disponible y configurado en las estaciones de trabajo.
- **Restricciones:**
 - La cobertura reportada por el frontend puede diferir levemente de acuerdo con las versiones de los paquetes NPM instalados en local.
 - Ciertos módulos de prueba dependen estrictamente de inyecciones de variables de entorno específicas para levantar contextos especiales.
 - Ventana de tiempo acotada a la duración estricta del Sprint académico (1 a 2 semanas máximo).

3. Comunicación de las Pruebas

3.1. Protocolo de Comunicación

- **Comunicación Interna:** Mensajería instantánea ágil vía WhatsApp para bloqueos urgentes, registro estructurado de tareas en el tablero Scrum de GitHub Projects y documentación técnica asíncrona mediante GitHub Issues.
- **Comunicación Externa:** Envío formal de informes técnicos de avance al docente a través del correo institucional o la plataforma académica oficial.
- **Resolución de Conflictos:** Mediación directa por parte del Test Lead en primera instancia; de no haber conciliación, se escalará formalmente con el Docente.

3.2. Puntos de Sincronización Estructurados

Punto de comunicación	Propósito	Frecuencia	Medios	Responsable	Audiencia
Inicio de Sprint	Planificación, distribución de módulos de análisis y asignación de tareas lógicas.	Una vez por Sprint	Google Meet / Presencial	Test Lead	Todo el equipo

Daily Stand-up	Seguimiento diario rápido de bloqueos y progreso en el setup de entornos.	Diario (15 min)	WhatsApp / Meet	WhatsApp / Meet	Todo el equipo
Reporte de Defectos	Registro de incompatibilidades de código o fallas de aserción encontradas.	Al detectar la falla	GitHub Issues		Test Lead / Architect
Revisión Docente	Registro de incompatibilidades de código o fallas de aserción encontradas.	2 veces por semana	Presencial / Meet	Test Lead	Docente + Equipo
Retrospectiva	Análisis del flujo de testing interno para aplicar mejoras de proceso.	Cierre del Sprint	Presencial / Meet	Test Lead	Todo el equipo
Informe de Cobertura	Publicación del reporte consolidado de coverage.py y HTML interactivo.	Post-ejecución local	GitHub Actions + Wiki	Test Architect	Todo el equipo

4. Registro de Riesgos Técnicos

N!	Riesgo Identificado	Probabilidad	Impacto	Severidad	Plan de Mitigación
1	Incompatibilidad menor en versiones de intérpretes	3	4	12	Forzar el uso estricto de las versiones del pipeline oficial

INVENTREE Plan de Pruebas Unitarias	UNSA - EPIS Página 12
--	--

	(Python/Node) al clonar la rama de InvenTree.				(Python 3.11 y Node 24) en todas las estaciones locales
2	Errores en la ejecución de la tarea de pruebas invoke dev.test por falta de dependencias del sistema operativo.	4	4	16	Ejecutar secuencialmente los comandos invoke dev.install y invoke update antes de levantar los tests. Consultar dependencias APT (gettext, poppler-utils).
3	Falsos negativos locales en la suite de pruebas que sí pasan exitosamente en el entorno de Actions.	3	3	9	Replicar con total exactitud las variables de entorno de CI en el archivo local de configuración (ej. INVENTREE_DB_ENGINE =sqlite3).
4	Degradación de las métricas de cobertura por debajo del umbral mínimo del 85% por alteración de fuentes.	2	4	8	Prohibir modificaciones estructurales sobre el código de la suite existente. Limitar el trabajo a re-ejecución y análisis de cobertura.
5	Cuellos de botella en los tiempos de entrega por la alta densidad de la suite (+1000 tests).	3	3	9	Segmentar las ejecuciones por submódulos prioritarios (part, stock) y activar ejecuciones en paralelo (--parallel).
6	Falta de comprensión sobre la arquitectura de modelos Django complejos para documentar los hallazgos.	3	3	9	Lectura obligatoria y previa de la guía de arquitectura oficial y revisión de los esquemas OpenAPI autogenerados.
7	Fallos masivos en los scripts de migración de base de datos (migration-tests) al usar motores locales pesados.	2	4	8	Aislar la suite de migraciones usando exclusivamente SQLite local, tal como se encuentra parametrizado en el workflow automatizado.

8	Retraso o desalineación de algún integrante del grupo en las asignaciones críticas del sprint.	2	4	6	Monitoreo riguroso en las reuniones Daily de 15 minutos; reasignación inmediata de tareas bloqueadas a través de GitHub Projects.
9	Agotamiento de la cuota gratuita de minutos de ejecución en los runners de GitHub Actions en el fork.	2	2	4	Restringir los push al repositorio remoto. Validar y pasar la suite localmente al 100% antes de subir código al fork.

5. Estrategia de Prueba Unitaria

5.1. Subprocesos de prueba

La suite se desglosa en tres subcapas de inspección backend y frontend de caja blanca:

5.1.1. Backend (Python/Django Core)

Basado en las clases estándar `django.test.TestCase` potenciado con el motor de ejecución de `pytest-django`. Cada una de las aplicaciones dentro de `src/backend/InvenTree/` encapsula su propio directorio aislado de pruebas:

- **Pruebas de Modelos:** Verificación estricta de constraints de base de datos, métodos lógicos internos de cálculo y propagación de disparadores lógicos (Django signals como `pre_save` y `post_save`).
- **Pruebas de API REST (DRF):** Pruebas construidas sobre `APITestCase` para validar la correcta deserialización de payloads, payloads inválidos, inyección de datos corruptos y aserciones de respuestas HTTP (200 OK, 201 Created, 400 Bad Request).
- **Lógica de Negocio y Permisos:** Validación de árboles recursivos multidimensionales (BOM de partes) y matrices de acceso según los tokens de rol (Admin, Operador, Lectura).

5.1.2. Frontend (Lógica Atómica)

Ejecutado a través del runner moderno Vitest. Las pruebas se limitan a componentes puros aislados del DOM:

INVENTREE Plan de Pruebas Unitarias	UNSA - EPIS Página 14
--	--

- Pruebas unitarias de funciones de transformación y parseo de datos de inventario.
- Control de estados iniciales y efectos en hooks lógicos personalizados de TypeScript.

5.1.3. Pruebas de Integración de API (Contexto de Suite)

Validación automatizada del ciclo completo de Request-Response interactuando de forma controlada con la base de datos de pruebas (CRUD completo, filtrado de parámetros mediante Query Params y paginación nativa).

5.2. Estructura Organizacional de la Suite

Ruta de Ubicación en el Repositorio	Framework Tecnológico	Descripción del Componente de Test
src/backend/InvenTree/*/tests/	Django TestCase + Pytest	Pruebas unitarias agrupadas por aplicación de negocio.
src/backend/InvenTree/InvenTree/tests.py	Django TestCase	
src/frontend/src/**/*.*.test.tsx	Vitest + React Testing Library	Pruebas globales del Core (Settings, versionamiento de API).
src/performance/	Pytest + CodSpeed	Pruebas unitarias de componentes de interfaz y lógica web.
test_stock_split	stock.tests	Pruebas automatizadas de regresión de rendimiento (Excluido de este hito).

5.3. Catálogo de Comandos de Automatización

Comandos mandatorios para la preparación y ejecución de la suite en entornos de desarrollo locales:

- Instalar dependencias del ecosistema:

```
invoke dev.install
```

- Ejecutar la suite completa del backend de forma directa:

```
invoke dev.test
```

- Ejecutar la suite recolectando reporte de cobertura de líneas:

```
invoke dev.test --coverage
```

- Ejecutar suite incorporando validación complementaria de linters:

```
invoke dev.test --check
```

- Compilar el catálogo de traducciones idiomáticas (Requerido pre-test):

```
invoke dev.translate
```

- Ejecutar exclusivamente los casos de prueba del módulo de Partes:

```
python manage.py test part
```

- Ejecutar tests de Stock con salida detallada por consola:

```
python manage.py test stock --verbosity=2
```

- Imprimir reporte simplificado de cobertura por módulos indicando líneas faltantes:

```
coverage report -m
```

- Compilar informe interactivo visual de cobertura en HTML:

```
coverage html
```

- Ejecutar la suite unitaria del cliente web frontend:

```
cd src/frontend && npm test
```

5.4. Variables de Entorno Obligatorias para Pruebas

Set de variables de entorno requeridas para forzar el comportamiento homogéneo de la suite en local, mapeadas directamente desde el job coverage de GitHub

Actions:

```
INVENTREE_DB_ENGINE=sqlite3
INVENTREE_DB_NAME=./inventree.sqlite
INVENTREE_PLUGINS_ENABLED=true
INVENTREE_DEBUG=true
INVENTREE_MEDIA_ROOT=/tmp/inventree_media
INVENTREE_STATIC_ROOT=/tmp/inventree_static
INVENTREE_ADMIN_USER=testuser
INVENTREE_ADMIN_PASSWORD=testpassword
INVENTREE_ADMIN_EMAIL=test@test.com
```

5.5. Técnicas de Diseño Aplicadas en la Suite

Las pruebas existentes de InvenTree implementan de manera implícita patrones matemáticos y lógicos de testing, los cuales analizaremos mediante las siguientes técnicas:

- **Partición de Equivalencia (PE):** Agrupamiento de entradas para evaluar bifurcaciones de código (ej. métodos `test_valid_part()` que inyectan datos sanos versus `test_invalid_part_name()` que fuerzan cadenas vacías o tipos erróneos).
- **Análisis de Valores Límite (AVL):** Pruebas unitarias detalladas que desafían los campos de caracteres o cantidades de stock numéricas justo en sus fronteras de desbordamiento de base de datos.
- **Tablas de Decisión (TD):** Mapeo de lógica condicional compleja, especialmente visible en los tests de permisos granulares donde se cruzan las variables: Rol de Usuario × Tipo de Operación (GET/POST) × Recurso de Inventario.
- **Transición de Estados:** Pruebas lógicas aplicadas a los flujos transaccionales de los modelos, asegurando que un objeto de stock mute correctamente su estado interno (`in_stock` → `allocated` → `used`).

5.6. Criterios de Finalización del Sprint

- Ejecución íntegra y sin fallos del 100% de la suite automatizada en los módulos core críticos (part, stock, order, build).
- Mantenimiento del indicador de cobertura global del backend en un porcentaje igual o superior al 87%, bloqueando cualquier regresión de líneas.

INVENTREE Plan de Pruebas Unitarias	UNSA - EPIS Página 17
--	--

- Cero (0) fallos o errores de aserción lógicos no controlados en el reporte de pytest.
- Documentación formal del procedimiento de réplica y carga de resultados en la GitHub Wiki del equipo.

5.7. Métricas de Control de Calidad

- Total de Tests Ejecutados: Conteo absoluto de casos de prueba corridos en el bloque backend.
- Porcentaje de Éxito:
- Tests PASSED/Total de tests ejecutados $\times 100$
- Tasa de Fallos: Cantidad neta de aserciones en estado FAIL o excepciones en ERROR.
- Porcentaje de Cobertura Global: Líneas ejecutadas/Total de líneas ejecutables $\times 100$
- Métricas de Cobertura Modular: Indicadores específicos calculados individualmente para las carpetas part/, stock/, order/, build/.
- Tiempo Total de Ejecución: Duración de reloj de la suite completa reportado por el motor de pytest.

6. Actividades y Estimados de Prueba

A continuación, se describen las actividades principales planificadas para el desarrollo del plan de pruebas unitarias:

N°	Actividad	Descripción
1	Configuración del Entorno Local	Clonar repo, instalar dependencias (invoke install), configurar variables de entorno, verificar que invoke dev.test funcione.
2	Lectura y Análisis de la Suite de Tests	Revisar la estructura de tests en cada módulo (part, stock, order, build, company, users, plugin). Identificar qué testa cada archivo.
3	Primera Ejecución Completa de Tests	Ejecutar 'invoke dev.test --check --coverage' y registrar resultados base (N° tests, cobertura, tiempo de ejecución, tests fallidos).
4	Análisis de Tests por Módulo Prioritario	Analizar en detalle los tests de: part/ (BOM, validaciones), stock/ (movimientos, ubicaciones), order/ (compra/venta), build/ (fabricación).

5	Documentación de Hallazgos en GitHub	Crear GitHub Issues para defectos encontrados. Actualizar GitHub Projects. Documentar todo en GitHub Wiki.
6	Segunda Ejecución y Verificación de Cobertura	Re-ejecutar suite completa. Generar reporte HTML de coverage. Comparar con primera ejecución.
7	Elaboración del Informe de Pruebas Unitarias	Redactar el informe formal de resultados siguiendo la plantilla del curso. Incluir capturas de pantalla del reporte de cobertura.
8	Revisión y Aprobación del Docente	Presentar los entregables al docente para revisión y feedback.

7. Requisitos y Herramientas del Entorno de Pruebas

7.1. Ambiente de Infraestructura Local

- **Sistemas Operativos Soportados:** Windows 10/11, Ubuntu 22.04 LTS o superior, macOS 12+.
- **Intérprete Python:** Versión exacta 3.11 (Requisito mandatorio mapeado desde el pipeline de Actions).
- **Entorno Node.js:** Versión de soporte estable Node.js 24.
- **Gestor de Control de Versiones:** Git 2.x.
- **Librerías del Sistema Linux Requeridas:** gettext (compilación de idiomas) y poppler-utils (renderizado lógico de PDFs de reporte).
- **Hardware Mínimo:** 4 GB de memoria RAM libre.

7.2. Frameworks y Herramientas Técnicas

- **Django TestCase (Django 4.x/5.x):** Clase base de aserciones del backend de InvenTree.
- **Pytest + Pytest-django:** Motor runner encargado de la orquestación ágil de la ejecución.
- **Coverage.py:** Librería técnica para auditar y pintar las líneas de código cubiertas.
- **DRF APITestCase:** Framework de inyección de clientes HTTP simulados para probar la API REST.
- **Vitest + React Testing Library:** Ecosistema encargado de la validación unitaria JS/TS.
- **Factory Boy:** Motor automatizado de instanciación dinámica de modelos lógicos (Fixtures ágiles).

INVENTREE Plan de Pruebas Unitarias	UNSA - EPIS Página 19
--	--